

CASE STUDY DOCUMENT

Expense Tracker with

Data Visualization

SpendSmart

Track. Visualize. Save. Grow.

Version 1.0 | 2026

1. Synopsis

SpendSmart is a full-stack Expense Tracker with Data Visualization that enables individuals to take complete control of their personal finances. Users can log daily expenses and income, organise transactions into custom categories, set per-category and overall monthly budgets, configure recurring transactions for automatic tracking, and gain deep financial insight through a rich dashboard of interactive charts and analytics.

The platform supports two primary roles: Users (who manage their own financial data) and Administrators (who oversee the platform). Users can track both expenses and income across unlimited custom categories with colour and icon customisation, monitor budget limits with real-time progress bars and configurable alert thresholds, visualise their finances through pie charts (category breakdown), bar charts (income vs expense), line charts (daily trend, savings rate), and a yearly overview. Advanced analytics features include a spending forecast using historical trend analysis, a Financial Health Score computed from savings rate and budget adherence, and a cash flow summary.

SpendSmart is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, expense tracking, income tracking, category management, budget management, analytics and reporting, recurring transaction automation, and notifications/alerts — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Users must register or log in (email/password or Google OAuth) to access their financial data.
- All financial data is strictly user-scoped; no user can access another user's transactions.
- Users can configure their preferred display currency and timezone.
- Role-based access control: User (full personal finance management) and Admin (platform oversight).
- In-app and email alerts are dispatched for budget overspends, recurring due dates, and monthly summary digests.
- Admin panel provides full user management, platform-wide analytics, and broadcast notification capability.

2.2 Authentication & Profile Requirements

- Register and log in via email/password or Google OAuth.
- Update profile including full name, avatar, email, and bio.
- Change password with current password verification.
- Set preferred display currency (e.g., USD, EUR, INR, GBP) — applied to all amounts displayed in the dashboard.
- Set timezone for correct date/time display of transactions.

- Set a monthly overall budget goal used in Financial Health Score computation.
- Deactivate account (soft delete preserving historical data).

2.3 Expense Management Requirements

- Add an expense with: title, amount, category, date, payment method (Cash/Card/UPI/Bank Transfer/Wallet), notes, and receipt attachment (image/PDF).
- Edit any existing expense: update any field without restriction.
- Delete an expense; all linked budget spend amounts are recalculated.
- View the full expense list with multi-criteria filtering: by category, date range, payment method, amount range, and keyword search.
- Filter expenses by a specific month/year combination for month-over-month review.
- Mark an expense as recurring to create a recurring rule (see Recurring Transaction requirements).
- Attach a receipt image or PDF to an expense for record-keeping.
- View total expenses for any selected period.

2.4 Income Management Requirements

- Add an income entry with: title, amount, category, source (Salary/Freelance/Business/Investment/Gift/Other), date, notes, and recurring flag.
- Edit or delete any income entry.
- View the full income list with filtering by source, date range, and keyword search.
- Mark income as recurring with a specified frequency (Daily, Weekly, Monthly, Quarterly, Yearly).
- View total income for any selected period.
- View a list of recurring income entries separately from one-time income.

2.5 Category Management Requirements

- Create custom expense and income categories with a name, type (EXPENSE/INCOME), colour code, and icon.
- Edit existing categories: update name, colour, icon.
- Delete a custom category (transactions in deleted categories revert to 'Uncategorised').
- System provides a set of default categories (Food, Transport, Shopping, Bills, Health, Entertainment, Salary, Investment, etc.) pre-created for every new user.
- Set a monthly budget limit per category (used for budget tracking and alerts).
- View all categories with their current month's spend vs. limit.

2.6 Budget Management Requirements

- Create budgets linked to a specific category and time period (Monthly, Weekly, or Custom date range).

- Set an alert threshold (%) at which a budget overspend warning notification is triggered (e.g., alert at 80% spent).
- View real-time budget progress: a progress bar showing spentAmount vs. limitAmount per budget.
- Budget spentAmount is updated automatically every time an expense in the matching category is added, edited, or deleted.
- Reset budget periods automatically at the start of each new period via a scheduled job.
- Receive in-app and email alerts when a budget reaches its configured threshold or is exceeded.
- View all active budgets with their progress at a glance on the dashboard.

2.7 Recurring Transaction Requirements

- Set up a recurring rule for any expense or income with: title, amount, type, category, frequency (Daily/Weekly/Monthly/Quarterly/Yearly), start date, optional end date, and payment method.
- The system automatically creates a transaction entry on each due date using a scheduled job.
- Upcoming due transactions for the current month are surfaced on the dashboard as a reminder widget.
- Edit a recurring rule: changes take effect from the next due date only (past auto-generated transactions are preserved).
- Deactivate a recurring rule to stop future auto-generation without deleting historical entries.
- Receive a notification reminder 3 days before each recurring transaction's due date.

2.8 Data Visualization Requirements

- Monthly Summary Dashboard: total income, total expenses, net savings, savings rate %, and budget utilisation summary for any selected month/year.
- Category Pie Chart: donut/pie chart showing expense distribution across categories for a selected period.
- Income vs. Expense Bar Chart: grouped or stacked bar chart comparing income and expense totals month-by-month for any trailing 12-month period.
- Daily Expense Trend Line Chart: daily expense totals plotted over the selected month for pattern identification.
- Savings Rate Trend Chart: monthly savings rate (%) plotted over the trailing 12 months.
- Yearly Overview: annual income, expense, and savings data displayed in a summary chart with monthly breakdown.
- Top Spending Categories: ranked list of top 5 categories by spend for any period.
- Cash Flow Summary: monthly inflow vs. outflow visualisation.
- Spending Forecast: projected next-month spend based on trailing 3-month average with trend adjustment.
- Financial Health Score: a 0–100 composite score computed from savings rate, budget adherence, and expense-to-income ratio.

2.9 Report Export Requirements

- Export expense/income data for any date range as a CSV file for use in external spreadsheets.
- Export a formatted monthly financial report as a PDF, including charts (rendered server-side), category summary, and transaction list.
- Receive a monthly financial summary email on the 1st of each month containing key metrics for the previous month.

2.10 Admin Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete.
- View all expenses and incomes platform-wide for compliance and audit.
- Access platform analytics: total users, total transactions, average monthly spend per user, most-used categories.
- View top-spending users and platform-wide spending trends.
- Send broadcast notifications to all users or by activity tier.
- Generate platform-level summary reports.
- View audit logs of all admin actions.

3. Use Case Diagram

The diagram below captures all actors and use cases within the SpendSmart platform — spanning authentication, expense and income management, category and budget configuration, recurring transaction automation, data visualization, financial insights, report export, notifications and alerts, and administrative oversight.

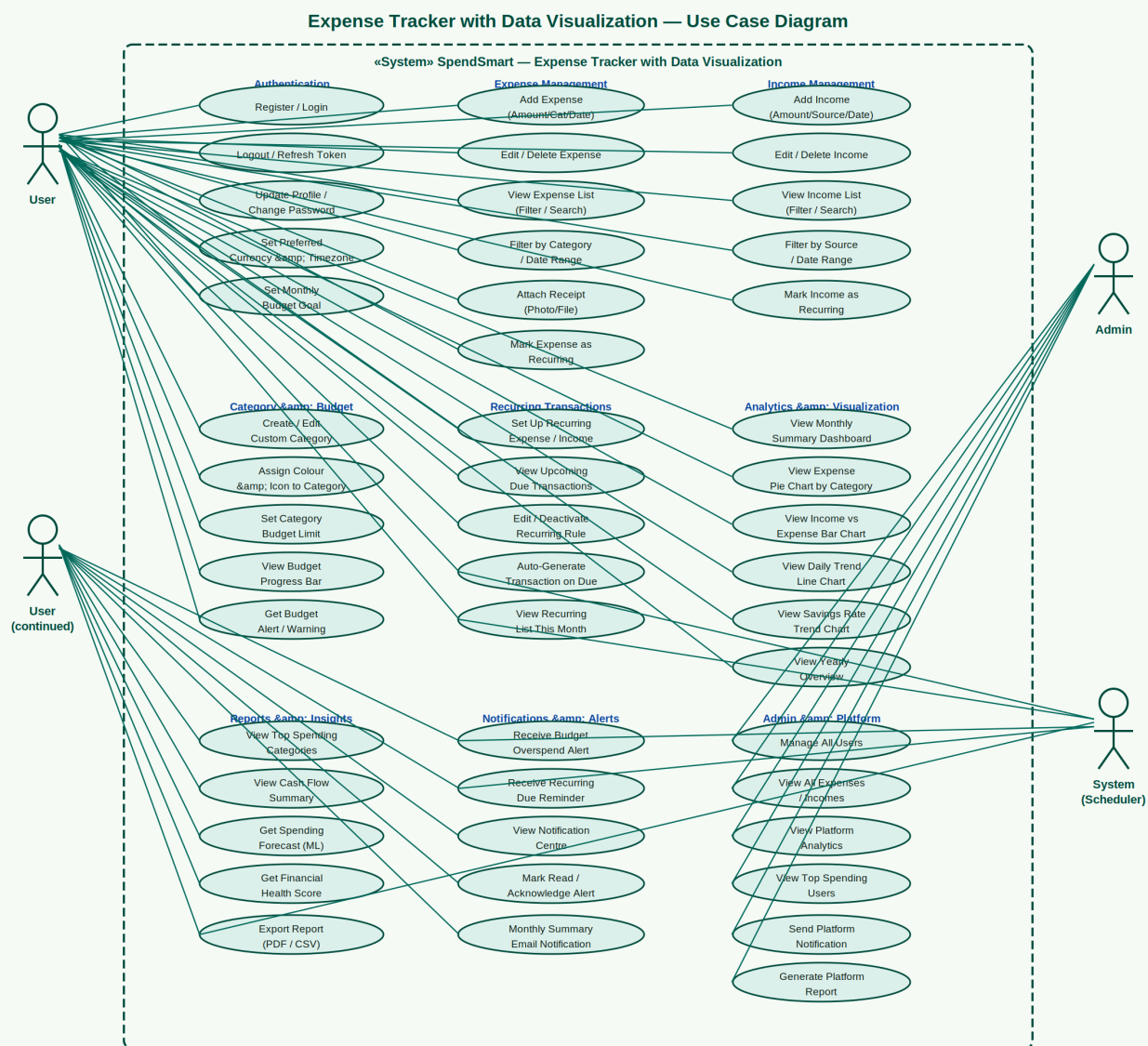


Figure 1: SpendSmart Expense Tracker with Data Visualization — Complete Use Case Diagram

3.1 Actors

Actor	Description
User	Registered individual who logs expenses and income, manages budgets, and views financial analytics
Admin	Platform administrator who manages users, views platform-wide data, and sends notifications
System (Scheduler)	Automated scheduler that generates recurring transactions, resets budget periods, and sends reminder/summary emails

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	User	Create account or authenticate via email or Google OAuth
Update Profile	User	Change name, email, avatar, timezone, or currency preference
Set Preferred Currency	User	Choose display currency applied to all financial summaries
Set Monthly Budget Goal	User	Define an overall monthly spending target for health score computation
Change Password	User	Update account password with current-password verification
Add Expense	User	Log an expense with amount, category, date, payment method, and optional receipt
Edit / Delete Expense	User	Modify or remove a logged expense; triggers budget recalculation
View Expense List	User	Browse all expenses with multi-criteria filter and keyword search
Filter by Category / Date	User	Narrow the expense list to a specific category or date range
Attach Receipt	User	Upload a photo or PDF of a payment receipt to any expense entry
Mark Expense as Recurring	User	Create a recurring rule from an existing expense entry
Add Income	User	Log an income entry with amount, source, category, and date
Edit / Delete Income	User	Modify or remove a logged income entry
View Income List	User	Browse all income entries with filter by source and date range

Use Case	Actor(s)	Description
Filter Income by Source	User	Narrow income list to a specific source (Salary, Freelance, etc.)
Mark Income as Recurring	User	Set a recurring rule for a regular income stream
Create / Edit Category	User	Add a custom category with name, type, colour, and icon
Assign Colour & Icon	User	Customise the visual appearance of any expense or income category
Set Category Budget Limit	User	Assign a monthly spending cap to a specific category
View Budget Progress	User	See a real-time progress bar for each active budget vs. its limit
Get Budget Alert	User, System	Receive in-app and email alert when budget threshold or limit is reached
Set Up Recurring Transaction	User	Create an automated rule for recurring expenses or income
View Upcoming Due	User	See all recurring transactions due within the current month
Edit / Deactivate Recurring	User	Modify or turn off a recurring rule for future periods
Auto-Generate Transaction	System	Scheduler creates a real expense/income entry on each recurring due date
View Recurring List	User	Browse all active recurring rules with their next due dates
View Monthly Summary Dashboard	User	See total income, expenses, net savings, and savings rate for any month
View Category Pie Chart	User	Interactive donut chart of expenses split by category for a period
View Income vs Expense Chart	User	Grouped bar chart comparing income and expenses month-by-month
View Daily Trend Chart	User	Line chart of daily expense totals across a selected month
View Savings Rate Trend	User	Line chart of monthly savings rate % over trailing 12 months
View Yearly Overview	User	Annual income, expense, and savings breakdown by month
View Top Spending Categories	User	Ranked list of categories by total spend for any period
View Cash Flow Summary	User	Monthly inflow vs. outflow visualisation in chart form

Use Case	Actor(s)	Description
Get Spending Forecast	User	AI-computed next-month spending estimate based on historical average
Get Financial Health Score	User	Composite 0–100 score measuring savings rate, budget adherence, and spend ratio
Export Report (PDF / CSV)	User	Download a formatted monthly report as PDF or raw data as CSV
Receive Budget Alert	User, System	In-app notification and email when a budget is nearing or exceeds its limit
Receive Recurring Reminder	User, System	Notification 3 days before a recurring transaction's due date
View Notification Centre	User	See all unread and past alerts with severity and entity deep-links
Mark Read / Acknowledge Alert	User	Dismiss individual or all notifications from the notification centre
Monthly Summary Email	System	Auto-sent email on the 1st of each month with prior month's key metrics
Manage All Users	Admin	View, suspend, reactivate, or permanently delete any user account
View All Expenses / Incomes	Admin	Browse platform-wide transaction data for compliance auditing
View Platform Analytics	Admin	Access aggregate KPIs: total users, transactions, avg spend per user
View Top Spending Users	Admin	Identify highest-spending users for platform health monitoring
Send Platform Notification	Admin	Broadcast in-app or email alerts to all or targeted users
Generate Platform Report	Admin	Export platform-level financial summary and user activity report

4. Class Diagrams — All Services

SpendSmart follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The ten class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service is the security and identity foundation of the platform. It manages registration, login, JWT token lifecycle, OAuth2 integration (Google), profile management, currency preference, monthly budget goal, and account deactivation. The currency and monthlyBudget fields on the User entity are referenced by the Analytics-Service when computing the Financial Health Score and formatting display amounts.

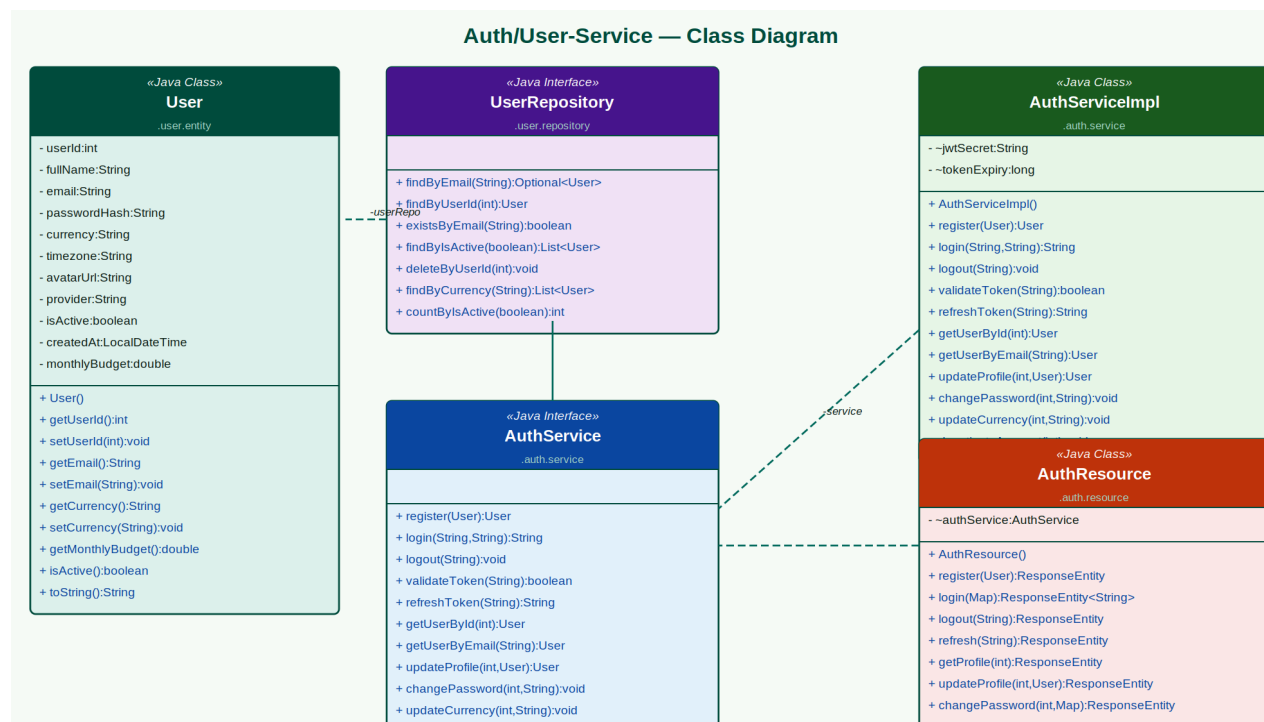


Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores <code>userId</code> , <code>fullName</code> , <code>email</code> , <code>passwordHash</code> , <code>currency</code> (ISO 4217), <code>timezone</code> (IANA), <code>avatarUrl</code> , <code>provider</code> (LOCAL/GOOGLE), <code>isActive</code> , <code>createdAt</code> , <code>monthlyBudget</code>
UserRepository	Java Interface	<code>findByEmail()</code> , <code>findById()</code> , <code>existsByEmail()</code> , <code>findByIsActive()</code> , <code>findByCurrency()</code> , <code>countByIsActive()</code> , <code>deleteById()</code>
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation/validation, OAuth2 handling, profile update, password change, currency update, account deactivation
AuthService	Java Interface	Declares <code>register()</code> , <code>login()</code> , <code>logout()</code> , <code>validateToken()</code> , <code>refreshToken()</code> , <code>getUserById()</code> ,

Component	Type	Responsibility
		updateProfile(), changePassword(), updateCurrency(), deactivateAccount()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/currency, /auth/deactivate

4.2 Expense-Service

The Expense-Service manages all expense transactions. Each Expense links a user to a category and captures amount, date, payment method, notes, receipt URL, and a recurring flag. On creation or update, the Budget-Service's spentAmount counter is incremented or decremented atomically. The service supports multi-criteria querying: by category, date range, month/year, type (EXPENSE/SPLIT), and keyword search across title and notes.

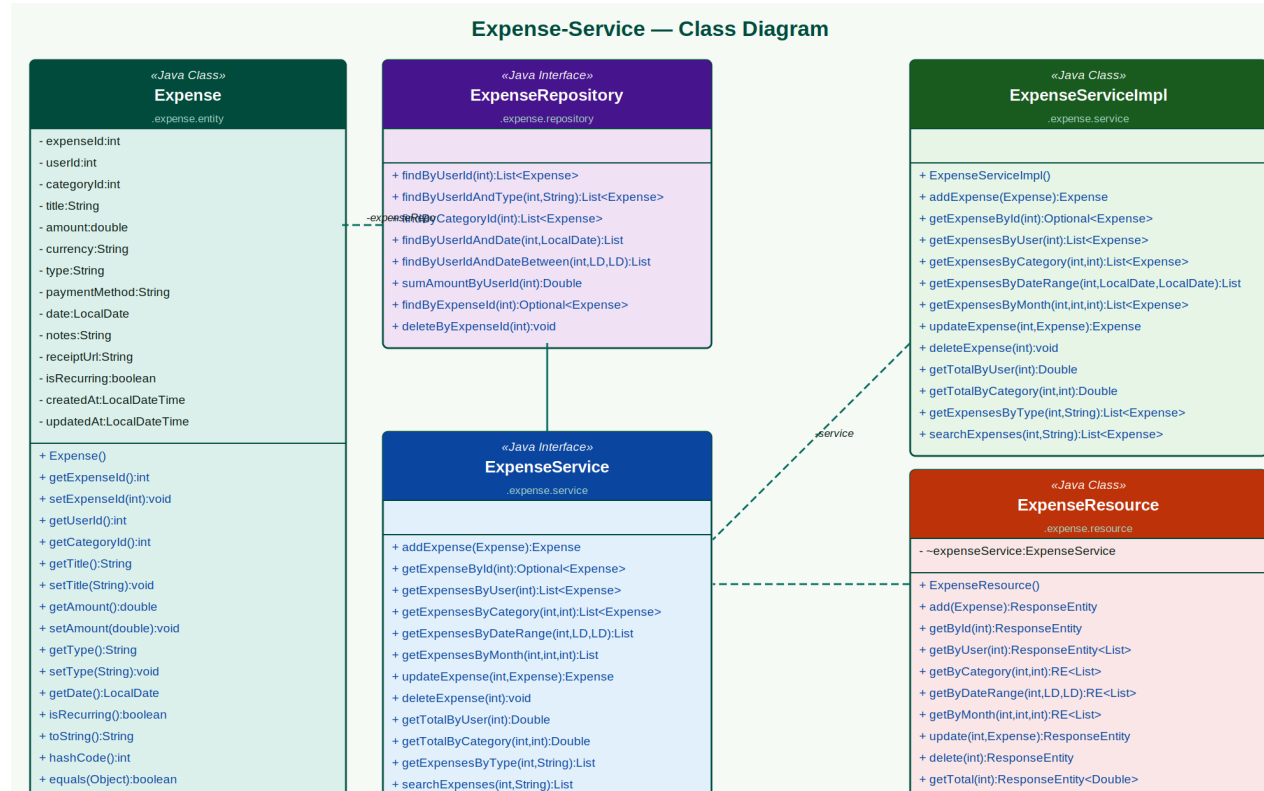


Figure 3: Expense-Service — Class Diagram

Component	Type	Key Attributes / Methods
Expense	Java Class (Entity)	expenseld, userid, categoryld, title, amount, currency, type, paymentMethod (CASH/CARD/UPI/BANK/WALLET), date, notes, receiptUrl, isRecurring, createdAt, updatedAt
ExpenseRepository	Java Interface	findById(), findByIdAndType(), findByIdAndCategoryld(), findByIdAndDate(), findByIdAndDateBetween(), sumAmountByUserId(), findByExpenseld(), deleteByExpenseld()
ExpenseServiceImpl	Java Class	addExpense(), getExpenseByld(), getExpensesByUser(), getExpensesByCategory(), getExpensesByDateRange(),

Component	Type	Key Attributes / Methods
		getExpensesByMonth(), updateExpense(), deleteExpense(), getTotalByUser(), getTotalByCategory(), getExpensesByType(), searchExpenses()
ExpenseService	Java Interface	Declares all expense CRUD, date-range/month queries, total aggregation, type filtering, and keyword search operations
ExpenseResource	Java Class (REST)	Exposes /expenses endpoints: POST (add), GET (by id/user/category/dateRange/month/type/search), PUT (update), DELETE, GET totals

4.3 Income-Service

The Income-Service manages all income entries. Each Income entry records the source (Salary, Freelance, Business, Investment, Gift, Other), which powers the income breakdown charts. A recurrencePeriod field specifies the frequency when isRecurring is true. The service exposes monthly and date-range queries consumed by the Analytics-Service for the income vs. expense bar chart and savings rate computation.

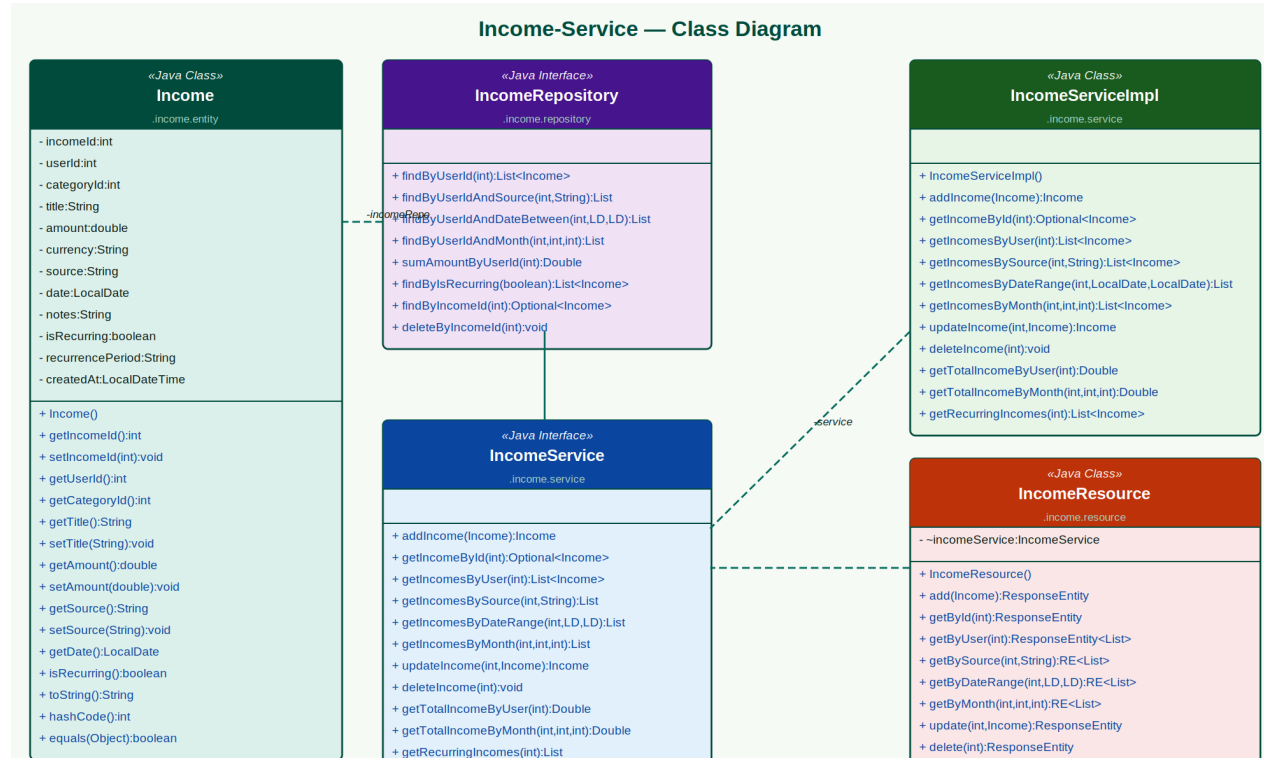


Figure 4: Income-Service — Class Diagram

Component	Type	Key Attributes / Methods
Income	Java Class (Entity)	incomeId, userId, categoryId, title, amount, currency, source (SALARY/FREELANCE/BUSINESS/INVESTMENT/GIFT/OTHER), date, notes, isRecurring, recurrencePeriod, createdAt
IncomeRepository	Java Interface	findById(), findByIdAndSource(), findByIdAndDateBetween(), findByIdAndMonth(), sumAmountByUserId(), findByIsRecurring(), findByIncomeId(), deleteByIncomeId()
IncomeServiceImpl	Java Class	addIncome(), getIncomeById(), getIncomesByUser(), getIncomesBySource(), getIncomesByDateRange(), getIncomesByMonth(), updateIncome(), deleteIncome(),

Component	Type	Key Attributes / Methods
		getTotalIncomeByUser(), getTotalIncomeByMonth(), getRecurringIncomes()
IncomeService	Java Interface	Declares all income CRUD, source/date-range/month queries, total aggregation, and recurring income retrieval operations
IncomeResource	Java Class (REST)	Exposes /incomes endpoints: POST (add), GET (by id/user/source/dateRange/month/recurring), PUT (update), DELETE, GET totals

4.4 Category-Service

The Category-Service manages the taxonomy of expense and income categories. Each Category is user-scoped (userId) and carries a visual identity (colorCode, icon) plus an optional budgetLimit for category-level budget tracking. On new user registration, a set of default system categories (Food, Transport, Bills, Health, Entertainment, Salary, etc.) is automatically seeded via `initDefaultCategories()`. The type field (EXPENSE/INCOME) ensures categories are only offered for the appropriate transaction type.

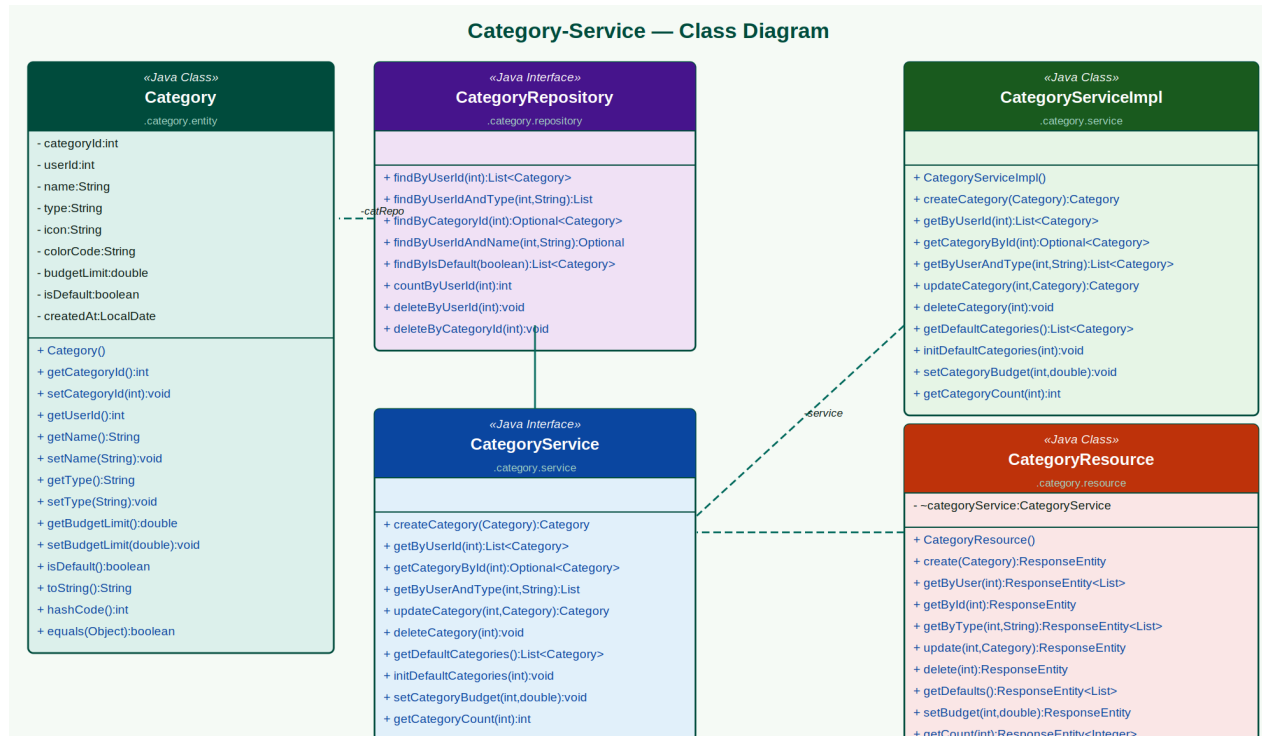


Figure 5: Category-Service — Class Diagram

Component	Type	Key Attributes / Methods
Category	Java Class (Entity)	categoryId, userId, name, type (EXPENSE/INCOME), icon (emoji or icon code), colorCode (hex), budgetLimit, isDefault, createdAt
CategoryRepository	Java Interface	findByUserId(), findByUserIdAndType(), findByCategoryId(), findByUserIdAndName(), findByIsDefault(), countByUserId(), deleteByUserId(), deleteByCategoryId()
CategoryServiceImpl	Java Class	createCategory(), getByUserId(), getCategoryById(), getByUserAndType(), updateCategory(), deleteCategory(), getDefaultCategories(), initDefaultCategories(), setCategoryBudget(), getCategoryCount()

Component	Type	Key Attributes / Methods
CategoryService	Java Interface	Declares all category CRUD, type-based filtering, default seeding, budget limit configuration, and count operations
CategoryResource	Java Class (REST)	Exposes /categories endpoints: POST (create), GET (by user/id/type/defaults), PUT (update/setBudget), DELETE, GET count

4.5 Budget-Service

The Budget-Service manages per-category and overall monthly budgets. Each Budget tracks spentAmount (updated atomically on every expense add/edit/delete via the Expense-Service) against a limitAmount, with a configurable alertThreshold percentage triggering a Notification-Service call when crossed. The getBudgetProgress operation returns a BudgetProgress DTO containing the percentage used, remaining amount, and alert status. A scheduled job resets budget periods at the start of each new period.

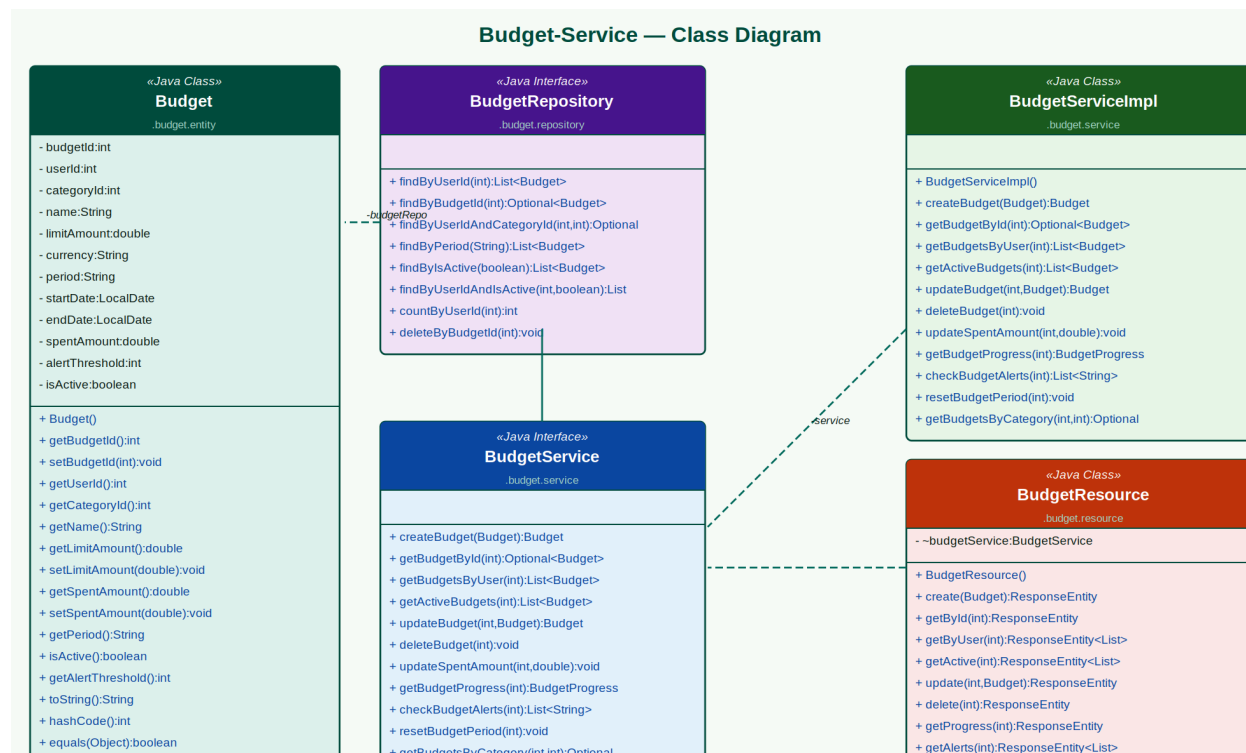


Figure 6: Budget-Service — Class Diagram

Component	Type	Key Attributes / Methods
Budget	Java Class (Entity)	budgetId, userId, categoryId, name, limitAmount, currency, period (MONTHLY/WEEKLY/CUSTOM), startDate, endDate, spentAmount, alertThreshold (%), isActive
BudgetRepository	Java Interface	findByUserId(), findByBudgetId(), findByUserIdAndCategoryId(), findByPeriod(), findByIsActive(), findByUserIdAndIsActive(), countByUserId(), deleteByBudgetId()
BudgetServiceImpl	Java Class	createBudget(), getBudgetById(), getBudgetsByUser(), getActiveBudgets(), updateBudget(), deleteBudget(), updateSpentAmount(), getBudgetProgress(),

Component	Type	Key Attributes / Methods
		checkBudgetAlerts(), resetBudgetPeriod(), getBudgetsByCategory()
BudgetService	Java Interface	Declares all budget CRUD, progress tracking, spent-amount update, alert checking, period reset, and active-budget query operations
BudgetResource	Java Class (REST)	Exposes /budgets endpoints: POST (create), GET (by user/id/active), PUT (update), DELETE, GET (progress/alerts), POST (resetPeriod)

4.6 Analytics / Report-Service

The Analytics/Report-Service is the data intelligence layer of the platform. It aggregates expense and income data from other services to produce the dashboard visualisation payloads. FinancialSnapshot records are written monthly to support trend analysis over time. The Financial Health Score (0–100) is a composite of savings rate (40%), budget adherence rate (40%), and expense-to-income ratio (20%). Spending forecasts use a 3-month trailing average with a momentum adjustment factor.

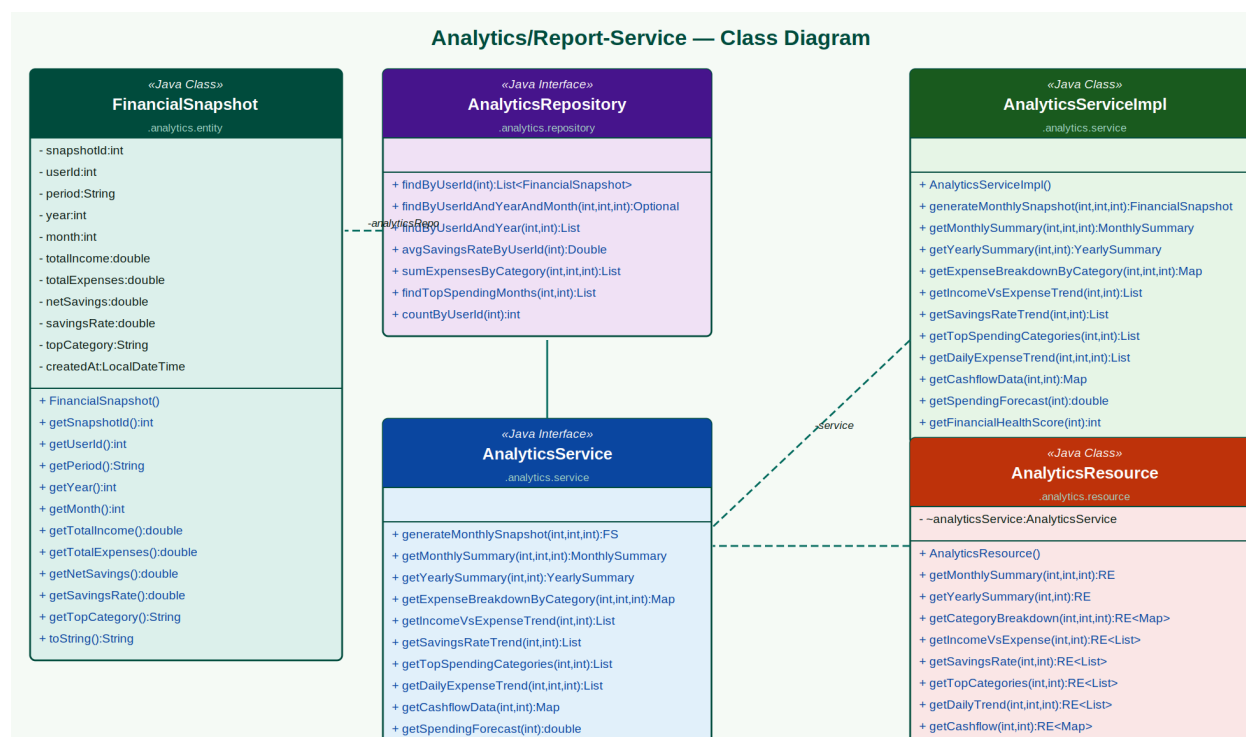


Figure 7: Analytics/Report-Service — Class Diagram

Component	Type	Key Attributes / Methods
FinancialSnapshot	Java Class (Entity)	snapshotId, userId, period, year, month, totalIncome, totalExpenses, netSavings, savingsRate, topCategory, createdAt — written monthly by scheduler
AnalyticsRepository	Java Interface	findByUserId(), findByUserIdAndYearAndMonth(), findByUserIdAndYear(), avgSavingsRateByUserId(), sumExpensesByCategory(), findTopSpendingMonths(), countByUserId()
AnalyticsServiceImpl	Java Class	generateMonthlySnapshot(), getMonthlySummary(), getYearlySummary(), getExpenseBreakdownByCategory(), getIncomeVsExpenseTrend(), getSavingsRateTrend(),

Component	Type	Key Attributes / Methods
		getTopSpendingCategories(), getDailyExpenseTrend(), getCashflowData(), getSpendingForecast(), getFinancialHealthScore()
AnalyticsService	Java Interface	Declares all snapshot generation, chart data retrieval, trend analysis, forecast, health score, and summary operations
AnalyticsResource	Java Class (REST)	Exposes /analytics endpoints: GET (monthlySummary, yearlySummary, categoryBreakdown, incomeVsExpense, dailyTrend, savingsRate, topCategories, cashflow, forecast, healthScore)

4.7 RecurringTransaction-Service

The RecurringTransaction-Service manages automated transaction rules. Each rule stores a frequency, nextDueDate, and an optional endDate. A Spring @Scheduled job runs daily at midnight to find all active recurring transactions whose nextDueDate equals today, generates the corresponding expense or income entry via the Expense/Income-Service, and advances the nextDueDate by one frequency period. Users receive a 3-day advance reminder notification via the Notification-Service.

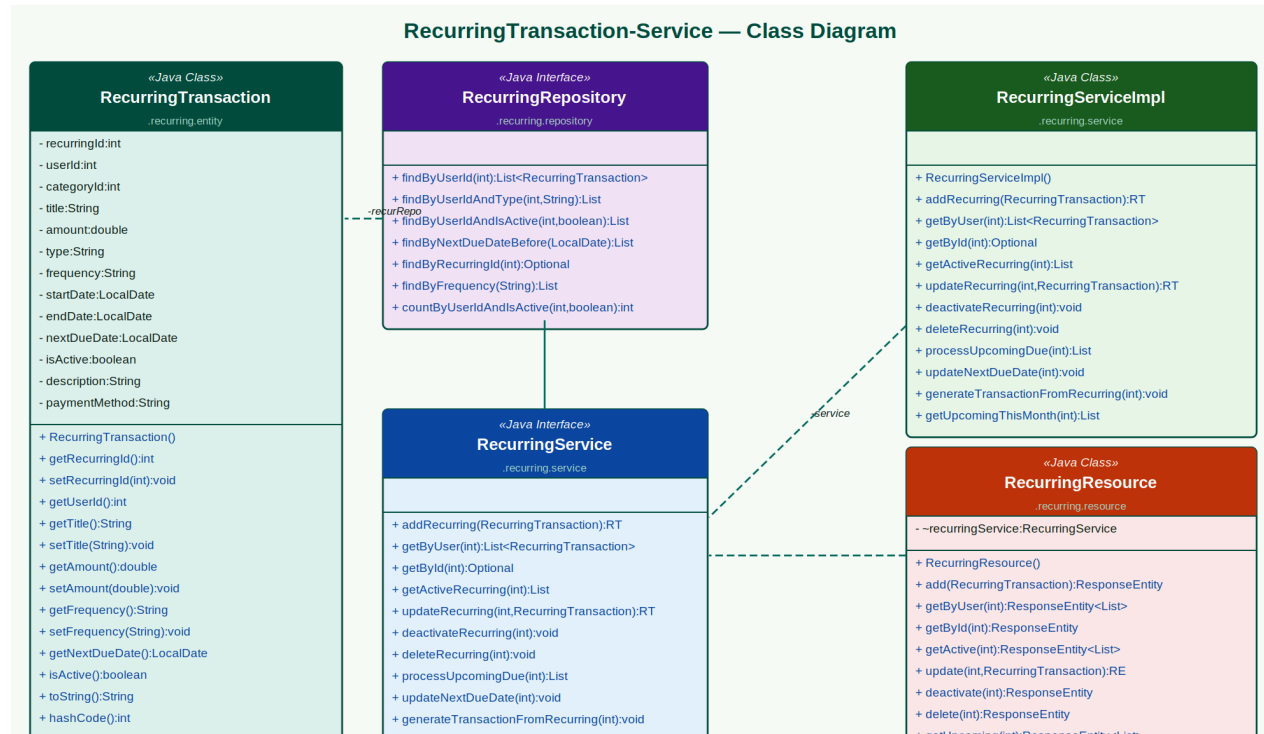


Figure 8: RecurringTransaction-Service — Class Diagram

Component	Type	Key Attributes / Methods
RecurringTransaction	Java Class (Entity)	recurringId, userId, categoryId, title, amount, type (EXPENSE/INCOME), frequency (DAILY/WEEKLY/MONTHLY/QUARTERLY/YEARLY), startDate, endDate, nextDueDate, isActive, description, paymentMethod
RecurringRepository	Java Interface	findByUserId(), findByUserIdAndType(), findByUserIdAndIsActive(), findByNextDueDateBefore(), findByRecurringId(), findByFrequency(), countByUserIdAndIsActive()
RecurringServiceImpl	Java Class	addRecurring(), getByUser(), getById(), getActiveRecurring(), updateRecurring(), deactivateRecurring(), deleteRecurring(), processUpcomingDue(), updateNextDueDate(),

Component	Type	Key Attributes / Methods
		generateTransactionFromRecurring(), getUpcomingThisMonth()
RecurringService	Java Interface	Declares all recurring rule CRUD, activation/deactivation, due-date processing, automatic transaction generation, and upcoming query operations
RecurringResource	Java Class (REST)	Exposes /recurring endpoints: POST (add), GET (by user/id/active/upcoming/upcomingMonth), PUT (update), DELETE (deactivate/delete)

4.8 Notification / Alert-Service

The Notification/Alert-Service manages all financial alerts and system notifications. It includes a domain-specific `sendBudgetAlert` operation triggered when Budget-Service detects a threshold breach. Notifications carry a severity (INFO/WARNING/CRITICAL) for visual prioritisation in the notification centre. CRITICAL budget alerts also trigger email dispatch. The service supports an acknowledgement flag in addition to the standard read flag, allowing users to confirm they have acted on an alert.

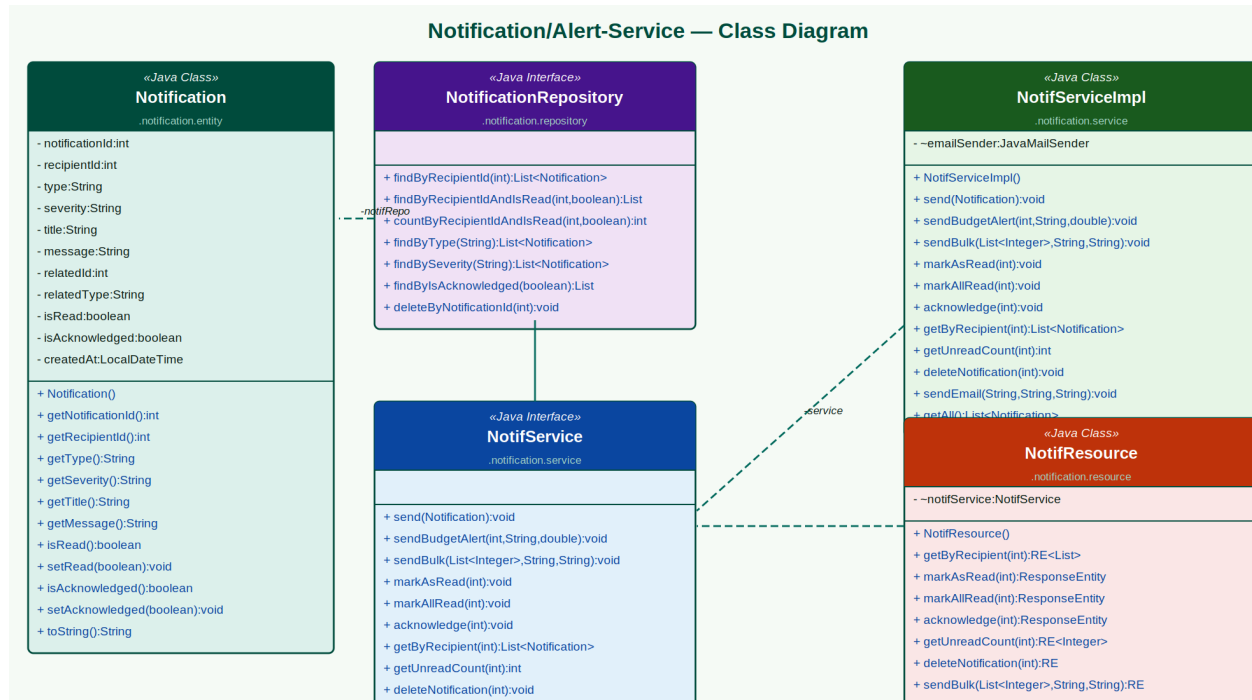


Figure 9: Notification/Alert-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, type (BUDGET_ALERT/RECURRING_DUE/MONTHLY_SUMMARY/BUDGET_EXCEEDED/SYSTEM), severity (INFO/WARNING/CRITICAL), title, message, relatedId, relatedType, isRead, isAcknowledged, createdAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findBySeverity(), findByIsAcknowledged(), deleteByNotificationId()
NotifServiceImpl	Java Class	send(), sendBudgetAlert(), sendBulk(), markAsRead(), markAllRead(), acknowledge(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), getAll()

Component	Type	Key Attributes / Methods
NotifService	Java Interface	Declares budget alert dispatch, bulk/single notification send, email dispatch, retrieval, read/acknowledge state, and deletion operations
NotifResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, acknowledge, getUnreadCount, delete, sendBulk, getAll

4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices and renders Thymeleaf views for users and administrators. It contains three controllers — ExpenseController (transaction management and categories), AnalyticsController (charts, budgets, and recurring transactions), and AdminController (admin panel) — each injecting relevant service dependencies via Spring @Autowired.

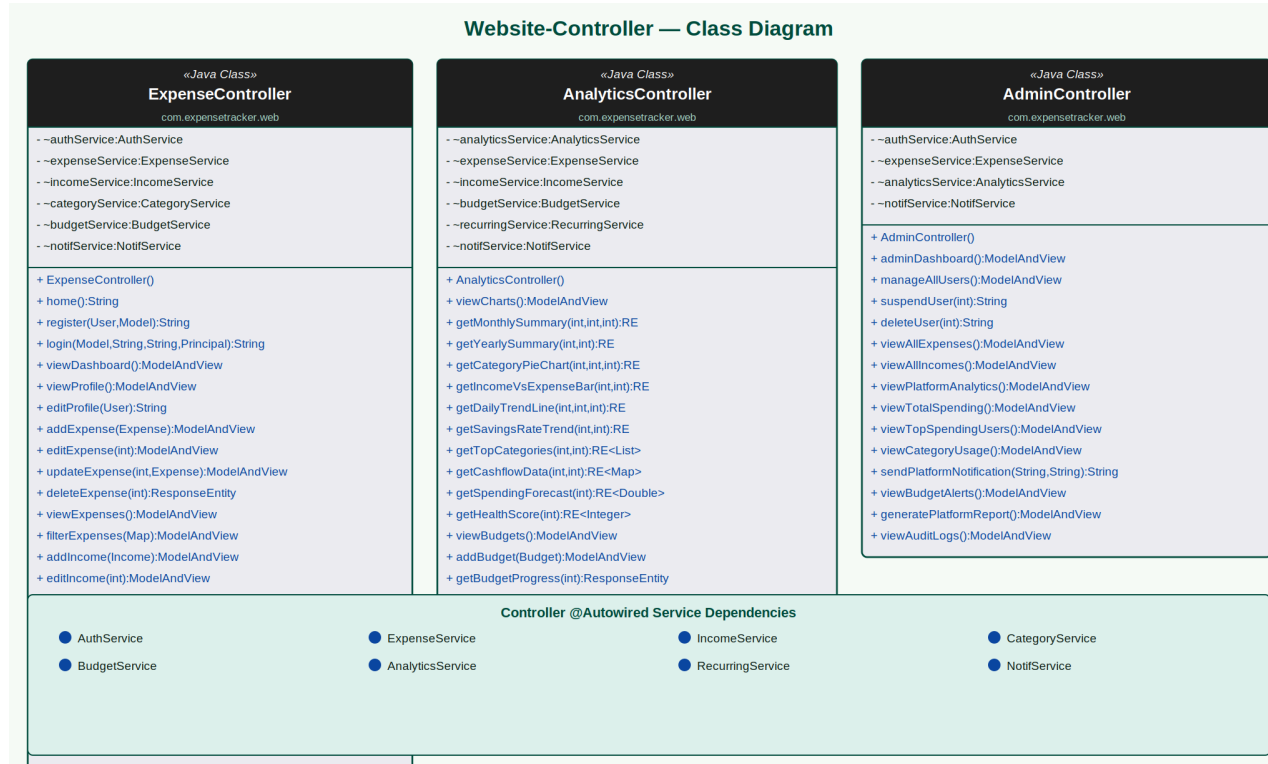


Figure 10: Website-Controller — Class Diagram

Controller	Type	Key Methods
ExpenseController	Java Class (MVC)	home(), register(), login(), viewDashboard(), viewProfile(), editProfile(), addExpense(), editExpense(), updateExpense(), deleteExpense(), viewExpenses(), filterExpenses(), addIncome(), editIncome(), updateIncome(), deleteIncome(), viewIncomes(), manageCategories(), addCategory(), setBudget(), viewNotifications(), logout()
AnalyticsController	Java Class (MVC)	viewCharts(), getMonthlySummary(), getYearlySummary(), getCategoryPieChart(), getIncomeVsExpenseBar(), getDailyTrendLine(), getSavingsRateTrend(), getTopCategories(), getCashflowData(), getSpendingForecast(), getHealthScore(), viewBudgets(), addBudget(),

Controller	Type	Key Methods
		getBudgetProgress(), getBudgetAlerts(), viewRecurring(), addRecurring(), getUpcoming()
AdminController	Java Class (MVC)	adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), viewAllExpenses(), viewAllIncomes(), viewPlatformAnalytics(), viewTotalSpending(), viewTopSpendingUsers(), viewCategoryUsage(), sendPlatformNotification(), viewBudgetAlerts(), generatePlatformReport(), viewAuditLogs()

5. Microservices Architecture Overview

SpendSmart is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous budget alerts, recurring transaction generation, and monthly summary emails are driven by RabbitMQ and Spring Scheduler.

Microservice	Base Package	Primary Domain
auth-service	com.spendsmart.auth	User accounts, JWT, OAuth2 (Google), currency/timezone preference
expense-service	com.spendsmart.expense	Expense CRUD, payment methods, receipt storage, keyword search
income-service	com.spendsmart.income	Income CRUD, source classification, recurring income tracking
category-service	com.spendsmart.category	Custom categories, default seeding, budget limit per category
budget-service	com.spendsmart.budget	Budget limits, spent tracking, progress, alert threshold management
analytics-service	com.spendsmart.analytics	Charts, summaries, forecasts, health score, financial snapshots
recurring-service	com.spendsmart.recurring	Recurring rules, scheduled auto-generation, due-date reminders
notification-service	com.spendsmart.notification	Budget alerts, recurring reminders, monthly digest, email/in-app
spendsmart-web	com.spendsmart.web	Spring MVC controllers, Thymeleaf view rendering, chart API proxy

6. Non-Functional Requirements

Category	Requirement
Performance	Dashboard chart data must load within 1.5 seconds; analytics queries are cached in Redis with a 10-minute TTL to reduce repeated aggregation cost
Scalability	Each microservice independently scalable via Docker/Kubernetes; Analytics-Service horizontally scaled for concurrent report generation
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for Google login; all API endpoints are user-scoped — cross-user data access is rejected at service layer
Data Isolation	Every repository query is parameterised with userId to ensure strict data isolation; no query can return data from a different user's account
Budget Atomicity	spentAmount updates on Budget are executed with database-level atomic increment/decrement to prevent inconsistency under concurrent expense additions
Recurring Reliability	Scheduled recurring job uses idempotency keys to prevent duplicate transaction generation on scheduler restarts
Forecast Accuracy	Spending forecast uses a weighted 3-month trailing average with an exponential smoothing factor; re-computed on each analytics request
Audit Trail	All admin actions (user suspension, deletion) and budget alert dispatches are logged with actor, timestamp, and entity reference
Export Quality	PDF reports render charts server-side using JFreeChart or Apache ECharts headless; CSV exports stream large datasets without memory overflow
Usability	Responsive dashboard on desktop and mobile; all charts are interactive (hover, zoom, filter); colour-blind-friendly palette option for chart themes

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, Google OAuth2
Database	MySQL (primary relational store per service); Redis (analytics cache, rate-limit counters, session store)
Charts / Frontend	Chart.js (interactive pie, bar, line charts) or Apache ECharts embedded in Thymeleaf; or React with Recharts for SPA
PDF Export	iText 7 or JasperReports for formatted monthly financial PDF reports with embedded chart images
CSV Export	Apache Commons CSV or OpenCSV for streaming large transaction datasets as CSV downloads
File Storage	AWS S3 for receipt image/PDF uploads; CloudFront CDN for fast delivery; multipart upload via Spring MultipartFile
Scheduled Jobs	Spring @Scheduled + Quartz Scheduler: daily recurring transaction job (midnight), monthly snapshot (1st of month), monthly digest email (1st of month), period reset job
Messaging	RabbitMQ for async budget alert dispatch, monthly summary email fan-out, and recurring reminder scheduling
Email	Spring JavaMailSender via AWS SES for budget alerts, recurring reminders, and monthly financial summary emails
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling with HPA on Analytics-Service
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints across all services
CI/CD	GitHub Actions pipeline: lint → unit test → integration test → Docker build → ECR push → Kubernetes rolling deploy

8. Glossary

Term	Definition
Expense	A financial outflow transaction logged by a user with amount, category, date, and payment method
Income	A financial inflow transaction logged by a user with amount, source, category, and date
Category	A user-defined or system-default grouping label for organising expenses and income into meaningful themes
Budget	A monetary spending limit defined for a category or overall, tracked against actual spending for a given period
Budget Progress	A real-time ratio of spentAmount to limitAmount for a budget, expressed as a percentage and displayed as a progress bar
Alert Threshold	A configurable percentage (e.g., 80%) at which a budget warning notification is automatically triggered
Recurring Transaction	An automated financial rule that creates a real expense or income entry on a defined schedule (daily/weekly/monthly/etc.)
nextDueDate	The next calendar date on which a recurring transaction will be automatically generated
Savings Rate	Monthly net savings divided by total income, expressed as a percentage: $(\text{Income} - \text{Expenses}) / \text{Income} \times 100$
Net Savings	The difference between total income and total expenses for a given period
Financial Health Score	A composite 0–100 score computed from savings rate, budget adherence rate, and expense-to-income ratio
Spending Forecast	A predicted next-month total expense amount computed using a weighted 3-month trailing average with trend adjustment
Cash Flow	The net difference between total inflows (income) and total outflows (expenses) over a time period
FinancialSnapshot	A point-in-time record of total income, expenses, net savings, and savings rate for a specific month, written monthly by the scheduler
Payment Method	The mechanism used to pay for an expense: Cash, Card, UPI, Bank Transfer, or Wallet
Income Source	The origin of an income entry: Salary, Freelance, Business, Investment, Gift, or Other